INTERNATIONAL JOURNAL FOR NUMERICAL METHODS IN ENGINEERING
Int. J. Numer. Meth. Engng 2014; **98**:418–444
Published online 3 March 2014 in Wiley Online Library (wileyonlinelibrary.com). DOI: 10.1002/nme.4637

Interface Jacobian-based Co-Simulation

S. Sicklinger[1,*,†], V. Belsky[2], B. Engelmann[2], H. Elmqvist[3], H. Olsson[3], R. Wüchner[1] and K.-U. Bletzinger[1]

[1]*Chair of Structural Analysis, Technische Universität München, Arcisstr. 21, 80333 München, Germany*
[2]*Dassault Systemes SIMULIA, USA*
[3]*Dassault Systemes CATIA, Sweden*

SUMMARY

Co-simulation is a prominent method to solve multi-physics problems. Multi-physics simulations using a co-simulation approach have an intrinsic advantage. They allow well-established and specialized simulation tools for different fields and signals to be combined and reused with minor adaptations in contrast to the monolithic approach. However, the partitioned treatment of the coupled system poses the drawback of stability and accuracy challenges. If several different subsystems are used to form the co-simulation scenario, these issues are especially important. In this work, we propose a new co-simulation algorithm based on interface Jacobians. It allows for the stable and accurate solution of complex co-simulation scenarios involving several different subsystems. Furthermore, the Interface Jacobian-based Co-Simulation Algorithm is formulated such that it enables parallel execution of the participating subsystems. This results in a high-efficient procedure. Furthermore, the Interface Jacobian-based Co-Simulation Algorithm handles algebraic loops as the co-simulation scenario is defined in residual form. Copyright © 2014 John Wiley & Sons, Ltd.

Received 21 May 2013; Revised 28 November 2013; Accepted 6 January 2014

KEY WORDS: co-simulation; *n*-code coupling; multi-code coupling; interface Jacobian; partitioned solution strategy; multi-physics; fluid-structure interaction

1. INTRODUCTION

Consumer products are becoming increasingly sophisticated and can be considered an assembly of different subsystems. Sophisticated simulation tools exist for solving individual and combined physical phenomena. Because of increased complexity, a designer must include more physics in his virtual model. Therefore, multi-physics simulations using a co-simulation approach have an intrinsic advantage, which allows well-established and specialized simulation tools for different fields to be combined and reused with minor adaptations in contrast to the monolithic approach. For the co-simulation approach, several solution algorithms are proposed in literature for the coupling of two simulators [1–3], but the full flexibility of the concept necessitates robust and efficient methods for the co-simulation of several codes with various interface coupling conditions [4]. This paper proposes a new methodology to enable the computational analysis of this class of problems.

The core idea of the Interface Jacobian-based Co-Simulation Algorithm (IJCSA) is to develop an algorithm that can handle a co-simulation scenario of an arbitrary number of codes where additional interface conditions can be specified. Another very important property of the algorithm is to feature accurate and stable simulations. On the other hand, performance is a critical issue. Therefore, the different simulators need to be able to run in parallel without data flow dependence.

*Correspondence to: S. Sicklinger, Technische Universität München, Arcisstr. 21, 80333 München, Germany.
†E-mail: stefan.sicklinger@tum.de

2. THE IDEA OF THE INTERFACE JACOBIAN-BASED CO-SIMULATION ALGORITHM

In order to illustrate the idea of the IJCSA on the basis of the well-known two-field coupling methodology, we start with an example where we have two subsystems each equipped with one input and one output quantity. Note that there are various synonyms used for subsystem in literature, for example, client, code, simulator, solver, and agent. The problem is then generalized to a multi-code scenario. The input and output relations for the problem are given by

$$Y_1 = \mathcal{S}_1(U_1), \tag{1}$$

$$Y_2 = \mathcal{S}_2(U_2). \tag{2}$$

Each of the subsystems $\mathcal{S}_1(U_1)$ and $\mathcal{S}_2(U_2)$ have state (internal) variables in addition to the output quantities Y_1 and Y_2. The state variables are referred to as X_1 and X_2.

The subsystems $\mathcal{S}_1(U_1)$ and $\mathcal{S}_2(U_2)$ are assumed to model a nonlinear relation between output $Y_\square$ and input $U_\square$ quantities, that is, $\mathcal{S}_\square$ is in general a nonlinear operator. The point of departure for the derivation of the IJCSA is the interface compatibility constraint equations

$$\mathcal{I}_1(Y_1, Y_2, U_1, U_2) = 0, \tag{3}$$

$$\mathcal{I}_2(Y_1, Y_2, U_1, U_2) = 0. \tag{4}$$

Where $\mathcal{I}_\square$ is the interface constraint operator, which can be also nonlinear in general. The interface constraint operator is essential to the IJCSA. It reflects the relations between input and output variables. The basic idea is to formulate a Newton method at interface level, where in contrast to a monolithic approach, a much smaller system needs to be solved. The Newton method is used to solve the set of in general nonlinear interface constraint equations. The combination of the two equation systems leads to

$$\mathcal{I}_1(\mathcal{S}_1(U_1), \mathcal{S}_2(U_2), U_1, U_2) = 0, \tag{5}$$

$$\mathcal{I}_2(\mathcal{S}_1(U_1), \mathcal{S}_2(U_2), U_1, U_2) = 0. \tag{6}$$

This is a nonlinear equation system. Hence, the Newton method should be applied to the latter system. Therefore, the interface residual is defined as

$$\mathcal{R}_1 = \mathcal{I}_1(\mathcal{S}_1(U_1), \mathcal{S}_2(U_2), U_1, U_2), \tag{7}$$

$$\mathcal{R}_2 = \mathcal{I}_2(\mathcal{S}_1(U_1), \mathcal{S}_2(U_2), U_1, U_2). \tag{8}$$

As a result of solving the set of equations (5) and (6), we determine the set of input values $U_\square$ for each system. The iteration sequence for the Newton method may be written as

$${}^{m+1}\boldsymbol{\phi} = {}^{m}\boldsymbol{\phi} - \mathcal{J}(\mathbf{r}({}^{m}\boldsymbol{\phi}))^{-1}\boldsymbol{r}({}^{m}\boldsymbol{\phi}), \tag{9}$$

which is equivalent to

$$\mathcal{J}(\mathbf{r}({}^{m}\boldsymbol{\phi}))\underbrace{({}^{m+1}\boldsymbol{\phi} - {}^{m}\boldsymbol{\phi})}_{\Delta^{m+1}\boldsymbol{\phi}} = -\mathbf{r}({}^{m}\boldsymbol{\phi}), \tag{10}$$

$$\mathcal{J}(\mathbf{r}({}^{m}\boldsymbol{\phi}))\Delta^{m+1}\boldsymbol{\phi} = -\mathbf{r}({}^{m}\boldsymbol{\phi}), \tag{11}$$

where $\boldsymbol{\phi}$ is the vector of unknowns, $\mathbf{r}$ is the residual vector, and $\mathcal{J}$ is the Jacobian operator. The dimension of the Jacobian operator is only dependent on the number of input variables at the interface level. The iteration index is denoted with m. For the example $\boldsymbol{\phi}$ and $\mathbf{r}$ are defined by

DOI: 10.1002/nme

$$\boldsymbol{\phi} = \begin{bmatrix} U_1 \\ U_2 \end{bmatrix}, \tag{12}$$

$$\mathbf{r} = \begin{bmatrix} \mathcal{R}_1 \\ \mathcal{R}_2 \end{bmatrix}. \tag{13}$$

The linear interface system of the Newton method can be written by

$$\begin{bmatrix} \frac{\partial \mathcal{R}_1}{\partial U_1} & \frac{\partial \mathcal{R}_1}{\partial U_2} \\ \frac{\partial \mathcal{R}_2}{\partial U_1} & \frac{\partial \mathcal{R}_2}{\partial U_2} \end{bmatrix} \begin{bmatrix} \Delta U_1 \\ \Delta U_2 \end{bmatrix} = - \begin{bmatrix} \mathcal{R}_1 \\ \mathcal{R}_2 \end{bmatrix}. \tag{14}$$

Let us assume the following simple interface conditions

$$\mathcal{I}_1(\mathcal{S}_2(U_2), U_1) = U_1 - Y_2 = 0, \tag{15}$$

$$\mathcal{I}_2(\mathcal{S}_1(U_1), U_2) = U_2 - Y_1 = 0. \tag{16}$$

with these interface conditions, equation system (14) becomes,

$$\begin{bmatrix} \mathrm{I} & -\frac{\partial \mathcal{S}_2}{\partial U_2} \\ -\frac{\partial \mathcal{S}_1}{\partial U_1} & \mathrm{I} \end{bmatrix} \begin{bmatrix} \Delta U_1 \\ \Delta U_2 \end{bmatrix} = - \begin{bmatrix} \mathcal{R}_1 \\ \mathcal{R}_2 \end{bmatrix}. \tag{17}$$

It is evident that the final form of the interface equation system is

$$\begin{bmatrix} \mathrm{I} & -\frac{\partial Y_2}{\partial U_2} \\ -\frac{\partial Y_1}{\partial U_1} & \mathrm{I} \end{bmatrix} \begin{bmatrix} \Delta U_1 \\ \Delta U_2 \end{bmatrix} = - \begin{bmatrix} \mathcal{R}_1 \\ \mathcal{R}_2 \end{bmatrix}. \tag{18}$$

Here $\frac{\partial Y_\square}{\partial U_\square}$ represents the derivative of the output with respect to the input.

2.1. The algorithm for two subsystems

The algorithm for the case of two subsystems is described in the chart Algorithm 2.1 on the basis of the previous considerations. Note that if the global Jacobian matrix J_{global} is set to the identity matrix, Algorithm 2.1 reduces to the classical fixed-point Jacobi scheme. For this case, it is in general necessary to introduce a relaxation parameter $\{\alpha \in \mathbb{R}\}$ to achieve convergence. This is shown in chart Algorithm 2.2.

2.2. Generalization of the concept

The case of multiple subsystems is presented below. All quantities are generalized to be vector quantities (bold notation). When an arbitrary number $r \geqslant 2$ of codes is present, the output–input relations for the i-th subsystem are defined as

$$\mathbf{Y}_i = \mathcal{S}_i(\mathbf{U}_i) \quad i = 1, ..., r. \tag{19}$$

The interface constraint operator for the i-th subsystem is defined as

$$\mathcal{I}_i(\mathbf{Y}_j, \mathbf{U}_j, j = 1, ..., r) \quad i = 1, ..., r. \tag{20}$$

Using Equation (19), one has

$$\mathcal{I}_i(\mathcal{S}_j(\mathbf{U}_j), \mathbf{U}_j, j = 1, ..., r) \quad i = 1, ..., r. \tag{21}$$

DOI: 10.1002/nme

Algorithm 2.1: Interface Jacobian-based Co-Simulation Algorithm for a two-code example.

```
   // Time loop
1  for n = 0 to n = n_end do
       // Iteration loop
2      for m = 0 to m = m_end do
           // Solve for all subsystems in parallel
3          ^m Y_1^{n+1} = S_1(^m U_1^n)
4          ^m Y_2^{n+1} = S_2(^m U_2^n)
           // Compute and check residual
5          ^m r^n = [^m R_1^n ; ^m R_2^n] = [^m U_1^n − ^m Y_2^{n+1} ; ^m U_2^n − ^m Y_1^{n+1}]
6          if ‖^m r^n‖ < ε then
7              break
           // Get part of interface Jacobian from each subsystem
8          ^m J_1^n = J(S_1(^m U_1^n)) = ^m ∂Y_1/∂U_1 ^n := ∂Y_1/∂U_1
9          ^m J_2^n = J(S_2(^m U_2^n)) = ^m ∂Y_2/∂U_2 ^n := ∂Y_2/∂U_2
           // Assemble global interface Jacobian
10         ^m J^n_global = A_J(^m J_1^n, ^m J_2^n) = [ 1 , −∂Y_2/∂U_2 ; −∂Y_1/∂U_1 , 1 ]
           // Solve for corrector
11         ^m J^n_global · ^m Δc^n = −^m r^n
           // Apply corrector
12         ^{m+1} U_1^n = ^m U_1^n + ^m Δc_1^n
13         ^{m+1} U_2^n = ^m U_2^n + ^m Δc_2^n
       // Initial solution for next time step
14     ^0 U_1^{n+1} = ^{m_end+1} U_1^n
15     ^0 U_2^{n+1} = ^{m_end+1} U_2^n
```

Algorithm 2.2: Fixed-point algorithm for a two-code example.

```
   // Time loop
1  for n = 0 to n = n_end do
       // Iteration loop
2      for m = 0 to m = m_end do
           // Solve all subsystems (parallel)
3          ^m Y_1^{n+1} = S_1(^m U_1^n)
4          ^m Y_2^{n+1} = S_2(^m U_2^n)
           // Compute and check residual
5          ^m r^n = [^m R_1^n ; ^m R_2^n] = [^m U_1^n − ^m Y_2^{n+1} ; ^m U_2^n − ^m Y_1^{n+1}]
6          if ‖^m r^n‖ < ε then
7              break
           // Apply update
8          ^{m+1} U_1^n = ^m U_1^n − α ^m R_1^n
9          ^{m+1} U_2^n = ^m U_2^n − α ^m R_2^n
       // Initial solution for next time step
10     ^0 U_1^{n+1} = ^{m_end+1} U_1^n
11     ^0 U_2^{n+1} = ^{m_end+1} U_2^n
```


DOI: 10.1002/nme

Hence, the interface constraint equation system is given by

$$\mathcal{I}_i(\mathcal{S}_j(\mathbf{U}_j), \mathbf{U}_j, j = 1, ..., r) = 0 \quad i = 1, ..., r. \tag{22}$$

The residual components of the residual vector are given by

$$\boldsymbol{\mathcal{R}}_i = \mathcal{I}_i(\mathcal{S}_j(\mathbf{U}_j), \mathbf{U}_j, j = 1, ..., r) \quad i = 1, ..., r. \tag{23}$$

The interface Jacobian system is now given with

$$\begin{bmatrix} \frac{\partial \mathcal{I}_1}{\partial \mathcal{S}_1}\frac{\partial \mathcal{S}_1}{\partial \mathbf{U}_1} + \frac{\partial \mathcal{I}_1}{\partial \mathbf{U}_1} & \cdots & \frac{\partial \mathcal{I}_1}{\partial \mathcal{S}_r}\frac{\partial \mathcal{S}_r}{\partial \mathbf{U}_r} + \frac{\partial \mathcal{I}_1}{\partial \mathbf{U}_r} \\ \vdots & \ddots & \vdots \\ \frac{\partial \mathcal{I}_r}{\partial \mathcal{S}_1}\frac{\partial \mathcal{S}_1}{\partial \mathbf{U}_1} + \frac{\partial \mathcal{I}_r}{\partial \mathbf{U}_1} & \cdots & \frac{\partial \mathcal{I}_r}{\partial \mathcal{S}_r}\frac{\partial \mathcal{S}_r}{\partial \mathbf{U}_r} + \frac{\partial \mathcal{I}_r}{\partial \mathbf{U}_r} \end{bmatrix} \begin{bmatrix} \Delta \mathbf{U}_1 \\ \vdots \\ \Delta \mathbf{U}_r \end{bmatrix} = - \begin{bmatrix} \boldsymbol{\mathcal{R}}_1 \\ \vdots \\ \boldsymbol{\mathcal{R}}_r \end{bmatrix}. \tag{24}$$

This is equivalent to

$$\begin{bmatrix} \frac{\partial \mathcal{I}_1}{\partial \mathbf{Y}_1}\frac{\partial \mathbf{Y}_1}{\partial \mathbf{U}_1} + \frac{\partial \mathcal{I}_1}{\partial \mathbf{U}_1} & \cdots & \frac{\partial \mathcal{I}_1}{\partial \mathbf{Y}_r}\frac{\partial \mathbf{Y}_r}{\partial \mathbf{U}_r} + \frac{\partial \mathcal{I}_1}{\partial \mathbf{U}_r} \\ \vdots & \ddots & \vdots \\ \frac{\partial \mathcal{I}_r}{\partial \mathbf{Y}_1}\frac{\partial \mathbf{Y}_1}{\partial \mathbf{U}_1} + \frac{\partial \mathcal{I}_r}{\partial \mathbf{U}_1} & \cdots & \frac{\partial \mathcal{I}_r}{\partial \mathbf{Y}_r}\frac{\partial \mathbf{Y}_r}{\partial \mathbf{U}_r} + \frac{\partial \mathcal{I}_r}{\partial \mathbf{U}_r} \end{bmatrix} \begin{bmatrix} \Delta \mathbf{U}_1 \\ \vdots \\ \Delta \mathbf{U}_r \end{bmatrix} = - \begin{bmatrix} \boldsymbol{\mathcal{R}}_1 \\ \vdots \\ \boldsymbol{\mathcal{R}}_r \end{bmatrix}. \tag{25}$$

2.3. The algorithm for an arbitrary number of subsystems

The general version of the algorithm is presented in the following chart (Algorithm 2.3).

Note that the assembly operator is denoted by $\mathcal{A}_{\mathcal{J}}$. From the derivations, it is obvious that the entries of the interface Jacobian matrix are composed of two basic kinds of derivatives. The first part needs to be provided by the interface $\frac{\partial \mathcal{I}_j}{\partial \mathbf{Y}_i}$ and $\frac{\partial \mathcal{I}_j}{\partial \mathbf{U}_i}$, and the second part needs to be provided by each individual subsystem $\frac{\partial \mathbf{Y}_i}{\partial \mathbf{U}_i}$.

2.4. Efficiency enhancements

The most time is generally spend for the solution of all subsystems in Algorithm 2.3 in line 3. This is in general a computationally expensive operation. The cost for the solution operation can be dramatically reduced by using an approximation.

During the extraction process of $\frac{\partial \mathbf{Y}_i}{\partial \mathbf{U}_i}$, a by-product is $\frac{\partial \mathbf{X}_i}{\partial \mathbf{U}_i}$, which is a derivative of the state variables with respect to the input variables. The derivative is used in the following approximation:

$${}^{m+1}\mathbf{X}_i \approx {}^{m}\mathbf{X}_i + {}^{m}\frac{\partial \mathbf{X}_i}{\partial \mathbf{U}_i}\,{}^{m}\Delta \mathbf{U}_i \tag{26}$$

For the output variables, this approximation is not used. The approximated state variables are used in combination with a nonlinear map g between the state and the output variables. A more general g may also depend on the input variables. That is

$${}^{m+1}\mathbf{Y}_i = g({}^{m+1}\mathbf{X}_i, {}^{m+1}\mathbf{U}_i). \tag{27}$$

This relation is the one that is used inside the simulators $\mathcal{S}_{\approx i}$.

Assuming that g is nonlinear, Algorithm 2.4 represents a more efficient version of Algorithm 2.3. Note that only line 3 of Algorithm 2.3 is modified.

$${}^{m}\mathbf{Y}_i^{n+1} \approx \mathcal{S}_{\approx i}({}^{m-1}\Delta \mathbf{c}_i^n, {}^{m}\mathbf{U}_i^n) \tag{28}$$

Equation (26) can be modified using the notation and indices in Algorithm 2.4

$${}^{m}\mathbf{X}_{\approx i} \approx {}^{m-1}\mathbf{X}_i + {}^{m-1}\frac{\partial \mathbf{X}_i}{\partial \mathbf{U}_i}\,{}^{m-1}\Delta \mathbf{c}_i^n \tag{29}$$

and for the output update

$${}^{m}\mathbf{Y}_i \approx g({}^{m}\mathbf{X}_{\approx i}, {}^{m}\mathbf{U}_i). \tag{30}$$

DOI: 10.1002/nme

Algorithm 2.3: Interface Jacobian-based Co-Simulation Algorithm.

// Time loop
for $n = 0$ *to* $n = n_{\text{end}}$ **do**
  // Iteration loop
  for $m = 0$ *to* $m = m_{\text{end}}$ **do**
    // Solve for all subsystems in parallel
    ${}^{m}\mathbf{Y}_i^{n+1} = \mathcal{S}_i({}^{m}\mathbf{U}_i^{n})$
    // Compute and check residual
    ${}^{m}\mathbf{r}^{n} = \begin{bmatrix} {}^{m}\mathcal{R}_1^{n} \\ \vdots \\ {}^{m}\mathcal{R}_r^{n} \end{bmatrix} = \begin{bmatrix} \mathcal{I}_1({}^{m}\mathbf{Y}_j^{n+1}, {}^{m}\mathbf{U}_j^{n}, j = 1, ..., r) \\ \vdots \\ \mathcal{I}_r({}^{m}\mathbf{Y}_j^{n+1}, {}^{m}\mathbf{U}_j^{n}, j = 1, ..., r) \end{bmatrix}$
    if $\|{}^{m}\mathbf{r}^{n}\| < \epsilon$ **then**
      break
    // Get part of interface Jacobian from each subsystem
    ${}^{m}\mathbf{J}_i^{n} = \mathcal{J}(\mathcal{S}_i({}^{m}\mathbf{U}_i^{n})) = {}^{m}\frac{\partial \mathbf{Y}_i}{\partial \mathbf{U}_i}{}^{n} := \frac{\partial \mathbf{Y}_i}{\partial \mathbf{U}_i}$
    // Assemble global interface Jacobian
    ${}^{m}\mathbf{J}_{\text{global}}^{n} = \mathcal{A}_{\mathcal{J}}({}^{m}\mathbf{J}_i^{n}, i = 1, ..., r) = \begin{bmatrix} \frac{\partial \mathcal{I}_1}{\partial \mathbf{Y}_1}\frac{\partial \mathbf{Y}_1}{\partial \mathbf{U}_1} + \frac{\partial \mathcal{I}_1}{\partial \mathbf{U}_1} & \cdots & \frac{\partial \mathcal{I}_1}{\partial \mathbf{Y}_r}\frac{\partial \mathbf{Y}_r}{\partial \mathbf{U}_r} + \frac{\partial \mathcal{I}_1}{\partial \mathbf{U}_r} \\ \vdots & \ddots & \vdots \\ \frac{\partial \mathcal{I}_r}{\partial \mathbf{Y}_1}\frac{\partial \mathbf{Y}_1}{\partial \mathbf{U}_1} + \frac{\partial \mathcal{I}_r}{\partial \mathbf{U}_1} & \cdots & \frac{\partial \mathcal{I}_r}{\partial \mathbf{Y}_r}\frac{\partial \mathbf{Y}_r}{\partial \mathbf{U}_r} + \frac{\partial \mathcal{I}_r}{\partial \mathbf{U}_r} \end{bmatrix}$
    // Solve for corrector
    ${}^{m}\mathbf{J}_{\text{global}}^{n} \cdot {}^{m}\Delta\mathbf{c}^{n} = -{}^{m}\mathbf{r}^{n}$
    // Apply corrector
    ${}^{m+1}\mathbf{U}_i^{n} = {}^{m}\mathbf{U}_i^{n} + {}^{m}\Delta\mathbf{c}_i^{n}$
  // Initial solution for next time step
  ${}^{0}\mathbf{U}_i^{n+1} = {}^{m_{\text{end}}+1}\mathbf{U}_i^{n}$

3. STABILITY CONSIDERATIONS

An undamped two-mass oscillator system is used to represent a coupled second-order IVP. The model problem is used to analyze the stability properties of the IJCSA and compare them with classical fixed-point algorithms [Gauss–Seidel (GS) and Jacobi (JC)]. The setting of the model problem is shown in Figure 1. The coupled system is decomposed in two domains (domain 1 and domain 2).

The interface of the two domains is formed by a massless rigid link between the masses m_1 and m_2. The properties of the partitioned systems are derived from the coupled quantities. The coupled

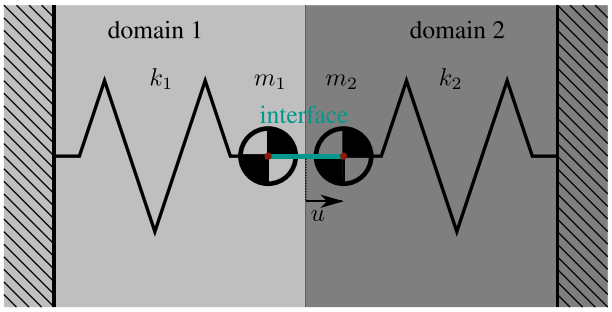


Figure 1. Model problem for stability considerations.

DOI: 10.1002/nme

Algorithm 2.4: Enhanced version of the IJCSA.

// Time loop
for $n = 0$ *to* $n = n_{\text{end}}$ **do**
  // Iteration loop
  for $m = 0$ *to* $m = m_{\text{end}}$ **do**
    if $m = 0$ **then**
      // Solve for all subsystems in parallel
      ${}^{m}\mathbf{Y}_i^{n+1} = \mathcal{S}_i({}^{m}\mathbf{U}_i^{n})$
    else
      // Approximated solve for all subsystems in parallel
      ${}^{m}\mathbf{Y}_i^{n+1} = \mathcal{S}_{\approx i}({}^{m-1}\Delta\mathbf{c}_i^{n}, {}^{m}\mathbf{U}_i^{n})$
    // Compute and check residual
    $${}^{m}\mathbf{r}^{n} = \begin{bmatrix} {}^{m}\mathcal{R}_1^{n} \\ \vdots \\ {}^{m}\mathcal{R}_r^{n} \end{bmatrix} = \begin{bmatrix} \mathcal{I}_1({}^{m}\mathbf{Y}_j^{n+1}, {}^{m}\mathbf{U}_j^{n},\ j = 1, ..., r) \\ \vdots \\ \mathcal{I}_r({}^{m}\mathbf{Y}_j^{n+1}, {}^{m}\mathbf{U}_j^{n},\ j = 1, ..., r) \end{bmatrix}$$
    if $\|{}^{m}\mathbf{r}^{n}\| < \epsilon$ **then**
      break
    // Get part of interface Jacobian from each subsystem
    $${}^{m}\mathbf{J}_i^{n} = \mathcal{J}(\mathcal{S}_i({}^{m}\mathbf{U}_i^{n})) = {}^{m}\frac{\partial \mathbf{Y}_i}{\partial \mathbf{U}_i}{}^{n} := \frac{\partial \mathbf{Y}_i}{\partial \mathbf{U}_i}$$
    // Assemble global interface Jacobian
    $${}^{m}\mathbf{J}_{\text{global}}^{n} = \mathcal{A}_{\mathcal{J}}({}^{m}\mathbf{J}_i^{n}, i = 1, ..., r) = \begin{bmatrix} \frac{\partial \mathcal{I}_1}{\partial \mathbf{Y}_1}\frac{\partial \mathbf{Y}_1}{\partial \mathbf{U}_1} + \frac{\partial \mathcal{I}_1}{\partial \mathbf{U}_1} & \cdots & \frac{\partial \mathcal{I}_1}{\partial \mathbf{Y}_r}\frac{\partial \mathbf{Y}_r}{\partial \mathbf{U}_r} + \frac{\partial \mathcal{I}_1}{\partial \mathbf{U}_r} \\ \vdots & \ddots & \vdots \\ \frac{\partial \mathcal{I}_r}{\partial \mathbf{Y}_1}\frac{\partial \mathbf{Y}_1}{\partial \mathbf{U}_1} + \frac{\partial \mathcal{I}_r}{\partial \mathbf{U}_1} & \cdots & \frac{\partial \mathcal{I}_r}{\partial \mathbf{Y}_r}\frac{\partial \mathbf{Y}_r}{\partial \mathbf{U}_r} + \frac{\partial \mathcal{I}_r}{\partial \mathbf{U}_r} \end{bmatrix}$$
    // Solve for corrector
    ${}^{m}\mathbf{J}_{\text{global}}^{n} \cdot {}^{m}\Delta\mathbf{c}^{n} = -{}^{m}\mathbf{r}^{n}$
    // Apply corrector
    ${}^{m+1}\mathbf{U}_i^{n} = {}^{m}\mathbf{U}_i^{n} + {}^{m}\Delta\mathbf{c}_i^{n}$
  // Initial solution for next time step
  ${}^{0}\mathbf{U}_i^{n+1} = {}^{m_{\text{end}}+1}\mathbf{U}_i^{n}$

stiffness is denoted by k, and the coupled mass is denoted by m. Hence, the eigenfrequency of the coupled system is given by $\omega = \sqrt{k/m}$.

With initial conditions $u(0) = 0$ and $\dot{u}(0) = 1$, the analytical solution of the coupled problem is given by $u(t) = {}^{1}/_{\omega}\sin(\omega t)$, where $u(t)$ is the time-dependent interface displacement. Let us introduce f being the force exerted by the mass m_2 onto m_1.

Furthermore, it is needed to define the quantities of the two subsystems m_1, m_2, k_1, and k_2. Therefore, the dimensionless parameters β_1 and β_2 are introduced where $\{\beta_1 \in \mathbb{R} \,|\, 0 \leqslant \beta_1 \leqslant 1\}$ and $\{\beta_2 \in \mathbb{R} \,|\, 0 \leqslant \beta_2 \leqslant 1\}$. Thus, for domain 1

$$m_1 = \beta_1 m, \tag{31}$$

$$k_1 = \beta_2 k, \tag{32}$$

and for domain 2, one has

$$m_2 = (1 - \beta_1)m, \tag{33}$$

$$k_2 = (1 - \beta_2)k. \tag{34}$$

DOI: 10.1002/nme

The governing ODEs for the two domains are as follows:

$$m_1\ddot{u}_1 + k_1 u_1 = f_1 \tag{35}$$

$$\beta_1 m\ddot{u}_1 + \beta_2 k u_1 = f_1 \tag{36}$$

$$m_2\ddot{u}_2 + k_2 u_2 = -f_2 \tag{37}$$

$$(1-\beta_1)m\ddot{u}_2 + (1-\beta_2)k u_2 = -f_2 \tag{38}$$

In operator notation, this is equivalent to

$$Y_1 = \mathcal{S}_1(U_1), \tag{39}$$

$$Y_2 = \mathcal{S}_2(U_2). \tag{40}$$

The interface compatibility constraint equations are

$$u_1 = u_2 = u, \tag{41}$$

$$f_1 = f_2. \tag{42}$$

Having that, the output and the input quantities are defined for the model problem as follows:

$$U_1 = f_1 \tag{43}$$

$$U_2 = u_2 \tag{44}$$

$$Y_1 = u_1 \tag{45}$$

$$Y_2 = f_2 \tag{46}$$

Note that this is a specific choice. It is in general possible to solve the same problem with different output/input combinations of the simulators. For instance, it is also possible to use only forces for the inputs of both simulators.

For the specific choice of the output and the input quantities, this is also called Dirichlet–Neumann decomposition, which in operator notation is given by

$$\mathcal{I}_1(\mathcal{S}_2(U_2), U_1) = U_1 - Y_2 = 0, \tag{47}$$

$$\mathcal{I}_2(\mathcal{S}_1(U_1), U_2) = U_2 - Y_1 = 0. \tag{48}$$

In order to be able to discuss numerical stability properties of the different coupling algorithms, a numerical time integrator needs to be deployed. As this is a model problem, the backward Euler (BE) time integrator is chosen; however, any time integrator could be taken for the discussion, in general. The BE method is used for both domains. Generally, different time integrators for the different domains may be present. The stability discussion is not limited to the particular choice of BE for both domains. However, it keeps the formulas to a reasonable level of complexity without harming the generality. The BE operator is given by

$$u^{n+1} = u^n + h\dot{u}^{n+1}, \tag{49}$$

$$\dot{u}^{n+1} = \dot{u}^n + h\ddot{u}^{n+1}, \tag{50}$$

where h is the time step. Rewriting these equations renders an approximation for the acceleration, namely,

$$\ddot{u}^{n+1} = \frac{1}{h^2}\left(u^{n+1} - 2u^n + u^{n-1}\right). \tag{51}$$

DOI: 10.1002/nme

With the help of this approximation, equations (36) and (38) can be discretized in time, which leads to

$$u_1^{n+1} = \frac{h^2}{\beta_1 m + \beta_2 k h^2} f_1^{n+1} + \frac{\beta_1 m}{\beta_1 m + \beta_2 k h^2} \left(2u_1^n - u_1^{n-1}\right), \tag{52}$$

$$f_2^{n+1} = \frac{\beta_1 m - m - k h^2 + \beta_2 k h^2}{h^2} u_2^{n+1} + \frac{(1-\beta_1)m}{h^2} \left(2u_2^n - u_2^{n-1}\right). \tag{53}$$

3.1. Gauss–Seidel fixed-point iterations

In fluid-structure interaction, the GS scheme is commonly used. There, it is also known as conventional serial staggered approach [5] in the context of loose coupling. Therefore, we analyze this pattern first. The communication pattern for two iterations is plotted in Figure 2. However, in a practical use case as many iterations are performed as needed in order to meet a certain convergence criteria. As we focus on iterative schemes in this work, iteration counters need to be added to Equations (52) and (53). Furthermore, the partitioned treatment requires that the time index of the input quantities has to be changed, which results in

$${}^m u_1^{n+1} = \frac{h^2}{\beta_1 m + \beta_2 k h^2} {}^m f_1^n + \frac{\beta_1 m}{\beta_1 m + \beta_2 k h^2} \left(2u_1^n - u_1^{n-1}\right), \tag{54}$$

$${}^m f_2^{n+1} = \frac{\beta_1 m - m - k h^2 + \beta_2 k h^2}{h^2} {}^m u_2^n + \frac{(1-\beta_1)m}{h^2} \left(2u_2^n - u_2^{n-1}\right). \tag{55}$$

By analyzing the GS communication pattern (see steps 3, 7, and 11 in Figure 2), it is obvious that the following holds:

$${}^m f_1^n = {}^m f_2^{n+1} \tag{56}$$

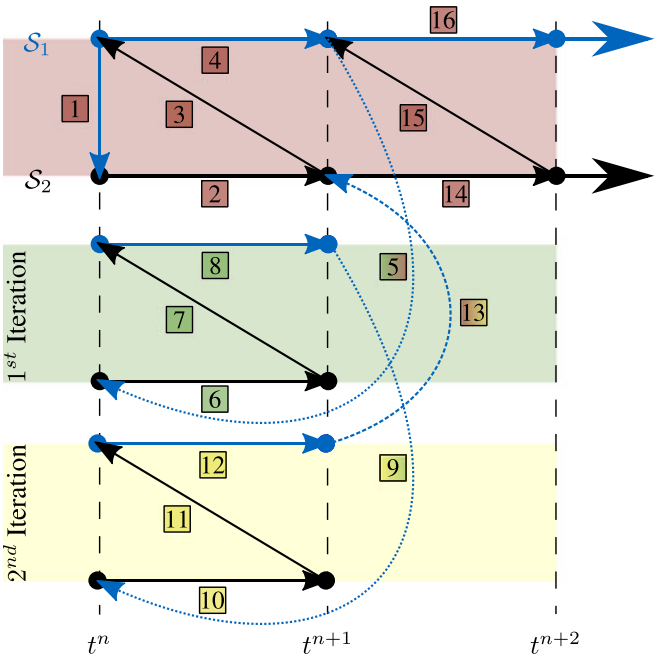


Figure 2. Iteration pattern for Gauss–Seidel.

DOI: 10.1002/nme

Therefore, Equations (54) and (55) can be coupled into one equation. After sorting the variables, one has

$$ {}^{m}u^{n+1} = \underbrace{\frac{\beta_1 m - m - kh^2 + \beta_2 kh^2}{\beta_1 m + \beta_2 kh^2}}_{A_{GS}} {}^{m}u^{n} + \frac{m}{\beta_1 m + \beta_2 kh^2}\left(2u^n - u^{n-1}\right), \tag{57} $$

where A_{GS} is the so-called convergence factor. Note that the operator notation for this problem can be simplified if the GS communication pattern is used. Hence, Equations (39) and (40) can be combined to

$$ Y_1 = \mathcal{S}_1(\mathcal{S}_2(U_2)). \tag{58} $$

For convergence of the GS iterations, the following must hold (see Theorem 7.2 [6]):

$$ |A_{GS}| < 1 \tag{59} $$

Note that for this particular model problem, an optimal relaxation parameter could be determined to provide the converged solution of the coupled problem after the first iteration. Including relaxation, the update rule is modified, and instead of using

$$ {}^{m+1}U_2 = {}^{m}Y_1, \tag{60} $$

the relaxation parameter $\{\alpha \in \mathbb{R}\}$ is incorporated in the update rule as follows:

$$ {}^{m+1}U_2 = {}^{m}U_2 - \alpha({}^{m}U_2 - {}^{m}Y_1) \tag{61} $$

The optimal relaxation parameter for the model problem is given by

$$ \alpha_{\text{optimal}} = \frac{1}{1 - A_{GS}}. \tag{62} $$

For more information, the reader is referred to page 767 of [7].

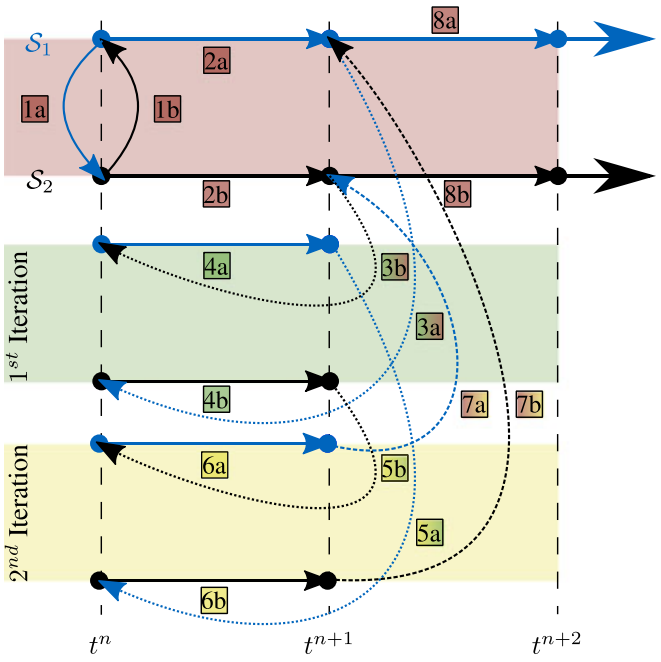


Figure 3. Iteration pattern for Jacobi.

DOI: 10.1002/nme

3.2. Jacobi fixed-point iterations

For the visual representation (Figure 3) of the iterative JC pattern, it is once more assumed that the iterations are converged after the second iteration. For an arbitrary multi-code scenario, the advantage of the JC pattern is that it allows for parallel execution of the subsystems. In Figure 3, this is indicated by a and b. For instance, the execution of $\mathcal{S}_1$ and $\mathcal{S}_2$ can be performed at the same time (steps 2a and 2b). In order to determine the convergence factor for the JC scheme, Equations (54) and (55) are written in the following matrix form:

$$\begin{bmatrix} {}^m u_1^{n+1} \\ {}^m f_2^{n+1} \end{bmatrix} = \underbrace{\begin{bmatrix} 0 & \frac{h^2}{\beta_1 m+\beta_2 k h^2} \\ \frac{\beta_1 m - m - k h^2 + \beta_2 k h^2}{h^2} & 0 \end{bmatrix}}_{\mathbf{A}_{\mathrm{JC}}} \begin{bmatrix} {}^m u_1^{n} \\ {}^m f_2^{n} \end{bmatrix} + \begin{bmatrix} \frac{\beta_1 m}{\beta_1 m+\beta_2 k h^2} \\ \frac{(1-\beta_1)m}{h^2} \end{bmatrix} \left(2u_2^n - u_2^{n-1}\right) \tag{63}$$

As the convergence factor $\mathbf{A}_{\mathrm{JC}}$ is not longer, a scalar the convergence criteria (59) need to be extended to a multi-dimensional space. In a multi-dimensional space, the spectral radius is introduced. Hence, the following equation must be satisfied for convergence.

$$\rho(\mathbf{A}_{\mathrm{JC}}) = \max_i(|\lambda_i|) < 1 \tag{64}$$

where λ_i are the eigenvalues of $\mathbf{A}_{\mathrm{JC}}$. Therefore, JC iterations will converge for the model problem if and only if the following condition is satisfied:

$$\sqrt{\left| \frac{h^2}{\beta_1 m + \beta_2 k h^2} \frac{\beta_1 m - m - k h^2 + \beta_2 k h^2}{h^2} \right|} < 1 \tag{65}$$

which may be further simplified to

$$\sqrt{\left| \frac{\beta_1 - 1 - \omega^2 h^2 + \beta_2 \omega^2 h^2}{\beta_1 + \beta_2 \omega^2 h^2} \right|} < 1, \tag{66}$$

which is the square root of the convergence factor for the GS pattern.

3.3. Interface Jacobian-based Co-Simulation Algorithm

As the IJCSA is applied to a linear problem, it needs one iteration to converge to the correct solution. In order to prove that the Newton method converges for the model problem, Theorem 2.12 of Deuflhard [8] is used. As the model problem is linear, the only assumption left to check is for which model properties $\mathbf{J}_{\mathrm{global}}$ is invertible.

$$\mathbf{J}_{\mathrm{global}} = \begin{bmatrix} 1 & -\frac{\partial Y_2}{\partial U_2} \\ -\frac{\partial Y_1}{\partial U_1} & 1 \end{bmatrix} \tag{67}$$

For the model problem, the interface Jacobian matrix is defined by

$$\mathbf{J}_{\mathrm{global}} = \begin{bmatrix} 1 & -\frac{\beta_1 m - m - k h^2 + \beta_2 k h^2}{h^2} \\ -\frac{h^2}{\beta_1 m+\beta_2 k h^2} & 1 \end{bmatrix} = \text{const.} \tag{68}$$

For a 2 x 2 matrix, the inverse is given by

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix}^{-1} = \frac{1}{ad - bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}. \tag{69}$$

DOI: 10.1002/nme

That means that the global interface Jacobian is not invertible if

$$\frac{\beta_1 m - m - kh^2 + \beta_2 kh^2}{h^2} \frac{h^2}{\beta_1 m + \beta_2 kh^2} = 1. \tag{70}$$

This is only the case if

$$m + kh^2 = 0, \tag{71}$$

which is not the case for a physical meaningful parameter set. Therefore, we conclude that the IJCSA is unconditionally stable for the model problem.

3.4. *Discussion of the stability properties*

In the following text, the results of the stability considerations for GS without relaxation, JC without relaxation and IJCSA will be discussed. One interesting limit case is $h \to 0$, this case represents the consistency of the algorithm. Thus, we have

$$\lim_{h\to 0} \rho(|\mathbf{A}_{\mathrm{JC}}|) = \sqrt{\left|\frac{\beta_1 - 1}{\beta_1}\right|}, \tag{72}$$

$$\lim_{h\to 0} (|A_{\mathrm{GS}})| = \left|\frac{\beta_1 - 1}{\beta_1}\right|. \tag{73}$$

For $h \to 0$ IJCSA is converging as long as $m > 0$.

As the convergence factor of the GS pattern is the square of one of the JC pattern, it is enough to analyze the convergence factor of the GS method. The limit of stability is the same for the JC and the GS method, the difference is in the rate of convergence. The smaller the convergence factor is, the faster the convergence rate is, that is, GS converges faster than the JC pattern. The convergence factor for GS may also be written as

$$|A_{\mathrm{GS}}| = \left|\frac{\beta_1 - 1 - \omega^2 h^2 + \beta_2 \omega^2 h^2}{\beta_1 + \beta_2 \omega^2 h^2}\right| = \rho(\mathbf{A}_{\mathrm{JC}})^2. \tag{74}$$

In Figure 4, the stability limits are shown for $\omega^2 h^2 = 0$, $\omega^2 h^2 = 1$, and $\omega^2 h^2 \to \infty$. As $\omega^2 h^2$ is increased, the stability limit is rotated around $\{\beta_1 = 0.5, \beta_2 = 0.5\}$. The lower left part is always

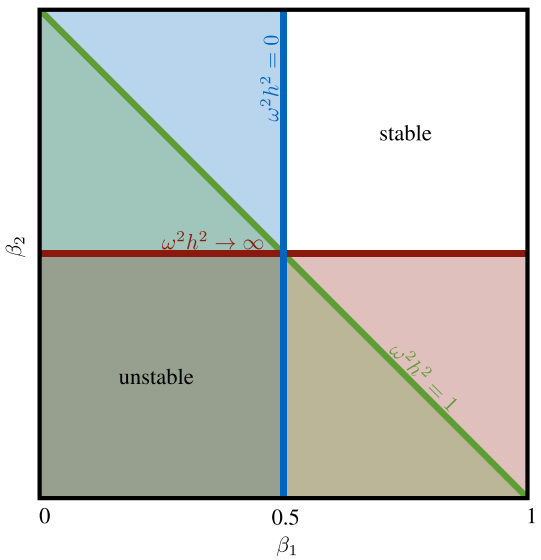


Figure 4. Stability limit graph.

DOI: 10.1002/nme

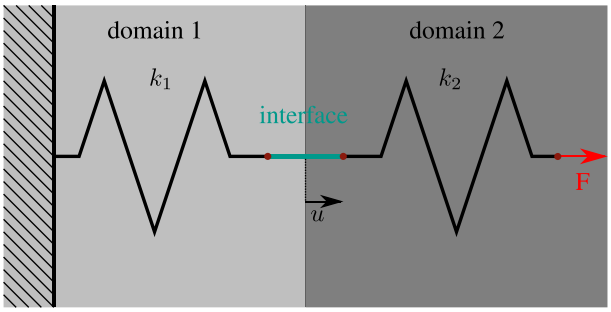


Figure 5. Model problem for decomposition considerations.

the unstable region. Thus, if $\beta_1 \in [0..0.5] \wedge \beta_2 \in [0..0.5]$, the parameter set is always unstable, independent of the choice of $\omega^2 h^2$.

At the consistency bounds, the stability is determined only by the mass ratio between domain 1 and domain 2. This is aligned with observations in fuild–structure interaction [9]. This shows that IJCSA is the best choice as it will render to coupled solution within one iteration, independent of any system parameter as long as they are physical.

Note that the stability is not only determined by the choice of the communication pattern (e.g. GS or JC), but also the choice of the decomposition is decisive. It can be shown that the IJCSA is unconditionally stable for the model problem independent of the choice of the decomposition. However, that is not the case for a general nonlinear problem. The best decomposition in terms of stability and efficiency is usually highly problem dependent. However, in general, the decomposition should not lead to an ill-posed subsystem. In order to demonstrate the consequences of a numerically unfavorable decomposition, a small example is presented in Figure 5. If the input of subsystem 2 is set to be a force f_2, subsystem 2 is not solvable anymore as it is missing a Dirichlet boundary condition.

4. EXAMPLES

The following section presents different examples that illustrate the performance of the IJCSA in more detail. The most general form of the IJCSA is presented in Algorithm 2.4. By analyzing this algorithm, three core capabilities of the subsystems can be identified. The most obvious capability of a 'black-box'-like subsystem is requested in line 4, which is a simple solution of one time step; we indicate this ability with `doSolve`. `doSolve` may indicate a method name in an implementation of the IJCSA. The core method of the IJCSA is given in line 10 where the subsystems need to deliver an interface Jacobian (`getInterfaceJacobian`). In line 6, the solution of all subsystems is performed. This can be performed in an efficient way by using approximation (26). Hence, the subsystem method may be called `doApproximatedSolve`.

These three core capabilities of the subsystems are illustrated in the following examples. All presented examples are nonlinear as most co-simulation scenarios involve nonlinear problems.

4.1. Truss versus truss problem

The IJCSA is used to solve a nonlinear transient structural mechanics problem. Within the problem, two truss systems are coupled. Each truss system is nonlinear because of the choice of the Hencky strain measure. The material law is chosen as St. Venant–Kirchhoff. A sketch of the problem setting is given in Figure 6.

The problem is similar to the one used in the stability section (Figure 1); therefore, we have

$$Y_1 = \mathcal{S}_1(U_1), \tag{75}$$

$$Y_2 = \mathcal{S}_2(U_2), \tag{76}$$

DOI: 10.1002/nme

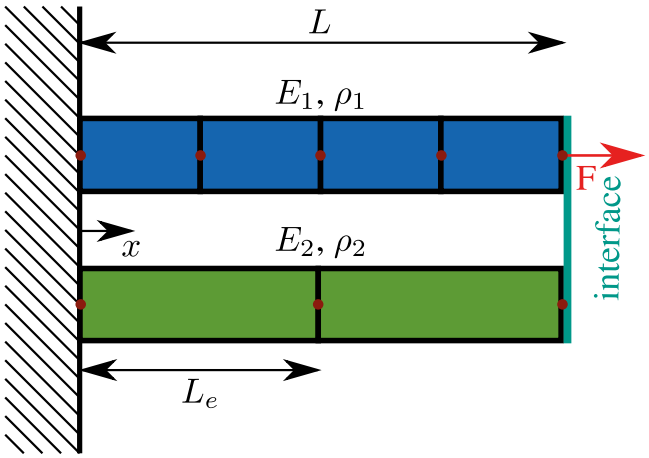


Figure 6. Setup of truss versus truss problem.

and the interface relations are given by

$$\mathcal{I}_1(\mathcal{S}_2(U_2), U_1) = U_1 - Y_2 = 0, \tag{77}$$

$$\mathcal{I}_2(\mathcal{S}_1(U_1), U_2) = U_2 - Y_1 = 0. \tag{78}$$

4.1.1. The 1D Hencky truss element. We start with the quadratic internal energy functional for the truss

$$\Pi_{\text{internal}} = \frac{1}{2}\int_V E\epsilon^2 \mathrm{d}V, \tag{79}$$

where E is the Young's modulus and ϵ is the Hencky strain. The weak form of Equation (79) reads

$$\delta\Phi_{\text{internal}} = \int_V E\epsilon(u(x))\delta\epsilon(u(x))\mathrm{d}V. \tag{80}$$

This may be split over a sum of finite elements (n_{e} being the total number of elements)

$$\delta\Pi_{\text{internal}} = \sum_{\text{e}=1}^{n_{\text{e}}} \delta\Pi_{\text{e}_{\text{internal}}}. \tag{81}$$

If linear shape functions are used for the approximation of the displacement field $u(x)$, the discretized weak form of one dimensional truss with Hencky strain measure is

$$\delta\Pi^{\text{h}}_{\text{e}_{\text{internal}}} = \underbrace{\begin{bmatrix} p_1(\mathbf{u}) \\ p_2(\mathbf{u}) \end{bmatrix}^T}_{\mathbf{p}_{\text{e}}(\mathbf{u})} \begin{bmatrix} \delta u_1 \\ \delta u_2 \end{bmatrix}, \tag{82}$$

where $\mathbf{p}_{\text{e}}(\mathbf{u})$ is called element internal force vector. Considering concentrated external forces only, we can write

$$\delta\Pi^{\text{h}} = \underbrace{\begin{bmatrix} p_1(\mathbf{u}) \\ p_2(\mathbf{u}) \end{bmatrix}^T}_{\mathbf{p}_{\text{e}}(\mathbf{u})} \begin{bmatrix} \delta u_1 \\ \delta u_2 \end{bmatrix} - \mathbf{f}_{\text{e}}{}^T \begin{bmatrix} \delta u_1 \\ \delta u_2 \end{bmatrix} = 0, \tag{83}$$

DOI: 10.1002/nme

where the element external force vector is denoted by $\mathbf{f}_e$. As this must be true for arbitrary test functions δu, the nonlinear equation system to be solved can be written as

$$\widetilde{\mathbf{r}}_{\text{subsystem}_e}(\mathbf{u}) = \begin{bmatrix} p_1(\mathbf{u}) \\ p_2(\mathbf{u}) \end{bmatrix} - \mathbf{f}_e = \mathbf{0}. \tag{84}$$

4.1.2. Dynamic extension of the 1D Hencky truss element. In order to expand the nonlinear residual (84), the d'Alembert forces must be added. Thus, the semi-discretized version of the dynamic nonlinear residual is

$$\mathbf{r}_{\text{subsystem}}(\mathbf{u}) = \mathbf{M}\ddot{\mathbf{u}} + \widetilde{\mathbf{r}}(\mathbf{u}) = \mathbf{M}\ddot{\mathbf{u}} + \mathbf{p}(\mathbf{u}) - \mathbf{f} = \mathbf{0}. \tag{85}$$

In order to discretize the vector of accelerations $\ddot{\mathbf{u}}$, the BE operator is used.

$$\ddot{\mathbf{u}}^{n+1} = \frac{1}{\Delta t^2}\left(\mathbf{u}^{n+1} - 2\mathbf{u}^n + \mathbf{u}^{n-1}\right) \tag{86}$$

Thus, Equation (85) can be written as follows:

$$\mathbf{r}_{\text{subsystem}}\left(\mathbf{u}^{n+1}\right) = \mathbf{M}\frac{1}{\Delta t^2}\left(\mathbf{u}^{n+1} - 2\mathbf{u}^n + \mathbf{u}^{n-1}\right) + \mathbf{p}\left(\mathbf{u}^{n+1}\right) - \mathbf{f} = \mathbf{0} \tag{87}$$

4.1.3. Solving the nonlinear equation system with the Newton method. The iteration sequence for the Newton method is given by

$${}_{l+1}\mathbf{u} = {}_{l}\mathbf{u} - \mathcal{J}(\mathbf{r}_{\text{subsystem}}({}_{l}\mathbf{u}))^{-1}\mathbf{r}_{\text{subsystem}}({}_{l}\mathbf{u}). \tag{88}$$

The Jacobian of the nonlinear functional $\mathcal{J}(\mathcal{D}({}_{l}\mathbf{u}))$ is given by

$$\mathcal{J}(\mathbf{r}_{\text{subsystem}}({}_{l}\mathbf{u})) = \underbrace{\mathbf{M}\frac{1}{\Delta t^2} + \underbrace{\mathcal{J}(\mathbf{p}(\mathbf{u}))}_{\mathbf{K}(\mathbf{u})}}_{\mathbf{A}(\mathbf{u})}, \tag{89}$$

where $\mathbf{K}(\mathbf{u})$ is referred to as tangent stiffness matrix and given by

$$\mathbf{K}(\mathbf{u}) = \mathcal{A}_{e=1}^{n_e}\mathbf{K}_e(\mathbf{u}), \tag{90}$$

here, $\mathcal{A}$ is the assembly operator, and $\mathbf{K}_e(\mathbf{u})$ is given by

$$\mathbf{K}_e(\mathbf{u}) = \begin{bmatrix} \frac{\partial p_1(\mathbf{u})}{\partial u_1} & \frac{\partial p_1(\mathbf{u})}{\partial u_2} \\ \frac{\partial p_2(\mathbf{u})}{\partial u_1} & \frac{\partial p_2(\mathbf{u})}{\partial u_2} \end{bmatrix}. \tag{91}$$

The element system matrices and vectors are summarized in Appendix A.1.

Next, the algorithms inside the subsystems for the truss versus truss problem are described in detail. Thus, the implementation of the IJCSA for the truss versus truss problem is illustrated.

Table I. Input/output quantities for subsystem 1.

Subsystem	Input U N	Output Y m
$\mathcal{S}_1$	$F_{\text{interface}}$	$u_{\text{interface}}$

DOI: 10.1002/nme

4.1.4. Subsystem 1 $Y_1 = \mathcal{S}_1(U_1)$. Subsystem 1 expects an interface force as an input (Table I). The subsystem additionally applies a constant force F to the truss system at the interface node (Figure 6).

The nonlinear residual function for subsystem 1 is given by

$$\mathbf{r}_{\text{subsystem}}\left(\mathbf{u}^{n+1}\right) = \mathbf{M}\frac{1}{\Delta t^2}\left(\mathbf{u}^{n+1} - 2\mathbf{u}^{n} + \mathbf{u}^{n-1}\right) + \mathbf{p}\left(\mathbf{u}^{n+1}\right) - \mathbf{f} = \mathbf{0}. \tag{92}$$

Please also note that the vector of unknowns is sorted such that the last DOF is the interface DOF. The internal degrees of freedom, in the following, are referred to as state variables $\mathbf{X}_1$. Therefore, Equation (92) becomes

$$\begin{aligned}\mathbf{r}_{\text{subsystem}}(\mathbf{u}^{n+1}) = \mathbf{M}\frac{1}{\Delta t^2}\Bigg(\underbrace{\begin{bmatrix}\left.\begin{matrix}u_1\\ u_2\\ \vdots\\ u_{i-1}\end{matrix}\right\}\mathbf{X_1}\\ u_{\text{interface}} = Y_1\end{bmatrix}}_{\mathbf{u}^{n+1}} - 2\mathbf{u}^{n} + \mathbf{u}^{n-1}\Bigg)\\ + \mathbf{p}(\mathbf{u}^{n+1}) - \underbrace{\begin{bmatrix}0\\0\\\vdots\\0\\F\end{bmatrix}}_{\mathbf{f}} + \underbrace{\begin{bmatrix}0\\0\\\vdots\\0\\F_{\text{interface}} = U_1\end{bmatrix}}_{\mathbf{f}_{\text{interface}}} = \mathbf{0}.\end{aligned} \tag{93}$$

`doSolve` The `doSolve` function of subsystem 1 expects an interface force as an input (U_1) and outputs the displacement of the interface DOF (Y_1). In order to solve the system of nonlinear equations (93) for every time step, the function applies a local Newton method to Equation (93)

$$\mathcal{J}\left(\mathbf{r}_{\text{subsystem}}({}_{l}\mathbf{u}^{n+1})\right) \quad \Delta_{l+1}\mathbf{u}^{n+1} = -\mathbf{r}_{\text{subsystem}}\left({}_{l}\mathbf{u}^{n+1}\right). \tag{94}$$

Note that l is the local Newton iteration counter and n is the time step counter.

`getInterfaceJacobian` The function `getInterfaceJacobian` computes the interface Jacobian and updates the internal Jacobian for the state variables. The bases for the following considerations is the so-called global sensitivity equation [10, chapter 7].

$$\frac{\partial \mathbf{r}_{\text{subsystem}}(\mathbf{u})}{\partial \mathbf{u}}\frac{\partial \mathbf{u}}{\partial \iota} = -\frac{\partial \mathbf{r}_{\text{subsystem}}}{\partial \iota} \tag{95}$$

In the case of the IJCSA, ι is represented via the interface input variable. For subsystem 1, this is the interface force $F_{\text{interface}}$. Hence, Equation (95) for subsystem 1 may be written as

DOI: 10.1002/nme

$$\begin{aligned}\mathbf{A}\frac{\partial \mathbf{u}}{\partial F_{\text{interface}}} &= -\frac{\partial \mathbf{r}_{\text{subsystem}}}{\partial F_{\text{interface}}},\\ \frac{\partial \mathbf{u}}{\partial F_{\text{interface}}} &= -\mathbf{A}^{-1}\frac{\partial \mathbf{r}_{\text{subsystem}}}{\partial F_{\text{interface}}}.\end{aligned} \tag{96}$$

Let us have a closer look at Equation (96)

$$\begin{bmatrix} \left.\begin{matrix} \frac{\partial u_1}{\partial F_{\text{interface}}} \\ \frac{\partial u_2}{\partial F_{\text{interface}}} \\ \vdots \\ \frac{\partial u_{i-1}}{\partial F_{\text{interface}}} \end{matrix}\right\} \frac{\partial \mathbf{X}_1}{\partial F_{\text{interface}}} \\ \left.\frac{\partial u_{\text{interface}}}{\partial F_{\text{interface}}}\right\} \frac{\partial Y_1}{\partial U_1} \end{bmatrix} = -\mathbf{A}^{-1}\begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \\ 1 \end{bmatrix}.$$

Thus, a Jacobian extraction involves a linear solution of the system. However, if within `doSolve` a direct solver is used, the decomposition of $\mathbf{A}$ is reused for the Jacobian extraction.

`doApproximatedSolve` The `doApproximatedSolve` method provides an efficient approximation for the solution of the subsystem

$${}^{m+1}\mathbf{X}_1 \approx {}^{m}\mathbf{X}_1 + {}^{m}\left|\frac{\partial \mathbf{X}_1}{\partial U_1}\right. {}^{m}\Delta U_1, \tag{97}$$

$${}^{m+1}\mathbf{X}_1 \approx {}^{m}\mathbf{X}_1 + {}^{m}\left|\frac{\partial \mathbf{X}_1}{\partial F_{\text{interface}}}\right. {}^{m}\Delta F_{\text{interface}}, \tag{98}$$

$${}^{m+1}Y_1 = g\left({}^{m+1}\mathbf{X}_1, {}^{m}U_1\right), \tag{99}$$

which does not involve a new integration in subsystem 1.

Note that the approximation is applied for the state variables only. Be aware that m is the iteration counter for the interface iterations. As the FEM is used for the discretization of this problem and the trial and test functions are chosen to be linear, the g function is simply a nonlinear relation between the displacements of the nodes $n-1$ and n.

4.1.5. Subsystem 2 $Y_2 = \mathcal{S}_2(U_2)$. In contrast to subsystem 1, subsystem 2 needs the interface displacement as input (Table II) .

The nonlinear residual function for subsystem 2 is given by

$$\mathbf{r}_{\text{subsystem}}\left(\mathbf{u}^{n+1}, \mathbf{f}_{\text{interface}}\right) = \mathbf{M}\frac{1}{\Delta t^2}\left(\mathbf{u}^{n+1} - 2\mathbf{u}^{n} + \mathbf{u}^{n-1}\right) + \mathbf{p}\left(\mathbf{u}^{n+1}\right) - \mathbf{f}_{\text{interface}} = \mathbf{0}. \tag{100}$$

Table II. Input/output quantities for subsystem 2.

Subsystem	Input U N	Output Y m
$\mathcal{S}_2$	$u_{\text{interface}}$	$F_{\text{interface}}$

DOI: 10.1002/nme

The internal degrees of freedom are in the following referred to as state variables $\mathbf{X}_2$ once more

$$\mathbf{r}_{\text{subsystem}}(\mathbf{u}^{n+1}, \mathbf{f}_{\text{interface}}) = \mathbf{M}\frac{1}{\Delta t^2}\left(\underbrace{\begin{bmatrix} \left.\begin{matrix} u_1 \\ u_2 \\ \vdots \\ u_{i-1} \end{matrix}\right\} \mathbf{X}_2 \\ u_{\text{interface}} = U_2 \end{bmatrix}}_{\mathbf{u}^{n+1}} - 2\mathbf{u}^n + \mathbf{u}^{n-1}\right) + \mathbf{p}\left(\mathbf{u}^{n+1}\right) - \underbrace{\begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \\ F_{\text{interface}} = Y_2 \end{bmatrix}}_{\mathbf{f}_{\text{interface}}} = \mathbf{0}. \tag{101}$$

`doSolve` The `doSolve` function of subsystem 2 expects an interface displacement as an input (U_2) and outputs the interface force of the interface DOF (Y_2). In order to solve the nonlinear equation system for every time step, the function applies an local Newton method to Equation (101)

$$\mathcal{J}\left(\mathbf{r}_{\text{subsystem}}({}_l\hat{\mathbf{u}}^{n+1})\right) \quad \Delta_{l+1}\hat{\mathbf{u}}^{n+1} = -\mathbf{r}_{\text{subsystem}}\left({}_l\hat{\mathbf{u}}^{n+1}\right). \tag{102}$$

Please note that $\mathbf{u}$ is replaced with $\hat{\mathbf{u}}$ because $u_{\text{interface}}$ is known now as it is an input quantity. Hence, $\hat{\mathbf{u}}$ is given by

$$\hat{\mathbf{u}} = \begin{bmatrix} u_1 \\ u_2 \\ \vdots \\ u_{i-1} \\ F_{\text{interface}} = Y_2 \end{bmatrix}. \tag{103}$$

`getInterfaceJacobian` Based on the Jacobian extraction procedure of subsystem 1 for subsystem 2, this is

$$\begin{aligned} \hat{\mathbf{A}}\frac{\partial \hat{\mathbf{u}}}{\partial u_{\text{interface}}} &= -\frac{\partial \mathbf{r}_{\text{subsystem}}}{\partial u_{\text{interface}}} \\ \frac{\partial \hat{\mathbf{u}}}{\partial u_{\text{interface}}} &= -\hat{\mathbf{A}}^{-1}\frac{\partial \mathbf{r}_{\text{subsystem}}}{\partial u_{\text{interface}}}. \end{aligned} \tag{104}$$

Equation (104) can be written as

$$\begin{bmatrix} \left.\begin{matrix} \frac{\partial u_1}{\partial u_{\text{interface}}} \\ \frac{\partial u_2}{\partial u_{\text{interface}}} \\ \vdots \\ \frac{\partial u_{i-1}}{\partial u_{\text{interface}}} \end{matrix}\right\} \frac{\partial \mathbf{X}_2}{\partial u_{\text{interface}}} \\ \left.\frac{\partial F_{\text{interface}}}{\partial u_{\text{interface}}}\right\} \frac{\partial Y_2}{\partial U_2} \end{bmatrix} = -\hat{\mathbf{A}}^{-1}\frac{\partial \mathbf{r}_{\text{subsystem}}}{\partial u_{\text{interface}}}.$$

Note that $\frac{\partial \mathbf{r}_{\text{subsystem}}}{\partial u_{\text{interface}}}$ is the last column of $\mathbf{A}$.

DOI: 10.1002/nme

doApproximatedSolve The doApproximatedSolve for subsystem 2 is similar to subsystem 1, namely,

$$ {}^{m+1}\mathbf{X}_2 \approx {}^{m}\mathbf{X}_2 + {}^{m}\left|\frac{\partial \mathbf{X}_2}{\partial U_2}\right. {}^{m}\Delta U_2, \tag{105} $$

$$ {}^{m+1}\mathbf{X}_2 \approx {}^{m}\mathbf{X}_2 + {}^{m}\left|\frac{\partial \mathbf{X}_2}{\partial u_{\text{interface}}}\right. {}^{m}\Delta u_{\text{interface}}, \tag{106} $$

$$ {}^{m+1}Y_2 = g\left({}^{m+1}\mathbf{X}_2, {}^{m}U_2\right). \tag{107} $$

4.1.6. Results. Numerical experiments are performed with the truss versus truss problem in order to demonstrate the performance of the IJCSA. Therefore, truss 1 is discretized with 20 elements and truss 2 with 10. The total length of the two truss systems is set to $L = 1$ m. Initial conditions are $u(0) = 0$ m and $\dot{u}(0) = 0$ m/s. The interface residual is considered as converged if

$$ \left\| {}^{m+1}\mathbf{r}^n \right\|_{\max} < 1.0E{-}10. \tag{108} $$

A constant external load $F = 4.25$ N is applied inside simulator 1. The local iteration convergence constraint for both subsystems is set to

$$ \left\| \mathbf{r}_{\text{subsystem}} \right\|_{\max} < 1.0E{-}12. \tag{109} $$

For the sake of clarity, this absolute measure of the residual is used. As the possible reference values, the applied force and the maximum interface displacement are close to one. This does not restrict the outcome of the discussion in anyway. Time discretization is performed with a step size $h = 0.01$ s and 200 time steps. To verify the coupled solution, the simulations are compared with a monolithic solution performed with Abaqus. It turned out that the IJCSA solution matches all the digits of the Abaqus solution. The densities are set to $\rho_1 = 0.55\ {}^{\text{kg}}/_{\text{m}^3}$ and $\rho_2 = 0.5\ {}^{\text{kg}}/_{\text{m}^3}$. The Young's moduli are $E_1 = 20\ {}^{N}/_{\text{m}^2}$ and $E_2 = 50\ {}^{N}/_{\text{m}^2}$. This parameter set is unstable with a standard JC pattern when no relaxation is applied. Therefore, constant under-relaxation $\alpha = 0.38$ is applied. The relaxation parameter was determined by numerical experiments and seems to be optimal for this problem parameter set. However, it is possible to also use adaptive relaxation methods. The Aitken relaxation method can be used [7] in combination with the GS pattern. Convergence for the Aitken method is only proven to be guaranteed if the residual is a scalar [11].

Table III presents the number of local and interface iterations for different algorithms. Classical fixed-point techniques and various versions of the IJCSA are compared. The number of local iterations is the most important one for the runtime. Each local Newton iteration of subsystem 1 and 2 involves a solution of a linear equation system. This is for the truss versus truss problem by far the most time consuming part in terms of the wall clock time. Hence, this is the measure for the efficiency of the different co-simulation algorithms.

As the subsystems use local iterations to solve the nonlinear state equations, an additional efficiency enhancement of the IJCSA may be performed. This is to converge the local iteration and the interface (outer) iterations together. It is about three times as fast as always converging the local iterations for the truss versus truss co-simulation problem.

If the problem is 'mildly' nonlinear, it might also be of advantage to use a modified Newton strategy in which Jacobian extraction is performed in the first iteration only within each time step. However, modified Newton methods exhibit in general only a linear convergence rate.

DOI: 10.1002/nme

Table III. Numerical results for the truss versus truss problem.

Description	No. of local iterations $\mathcal{S}_1$	No. of local iterations $\mathcal{S}_2$	No. of local iterations $\mathcal{S}_1$ (approx)	No. of local iterations $\mathcal{S}_2$ (approx)	No. of interface iterations
Aitken relaxation with Gauss–Seidel pattern	4157	3720	0	0	1092
Constant under-relaxation with Jacobi pattern	46388	44333	0	0	17446
Basic IJCSA (Algorithm 2.3)	2526	2568	0	0	785
Enhanced IJCSA (Algorithm 2.4)	771	807	1619	1178	789
Basic IJCSA (Algorithm 2.3) with a fixed number of local Newton iterations for the solve step (1 local iteration)	1146	1146	0	0	1146
Enhanced IJCSA (Algorithm 2.4) with a fixed number of local Newton iterations for the solve step (1 local iteration)	200	200	1262	932	666
Enhanced IJCSA (with Jacobian extraction in the first iteration within each time step only (modified Newton)) (1 local iteration)	200	200	1854	1400	900

IJCSA, Interface Jacobian-based Co-Simulation Algorithm.

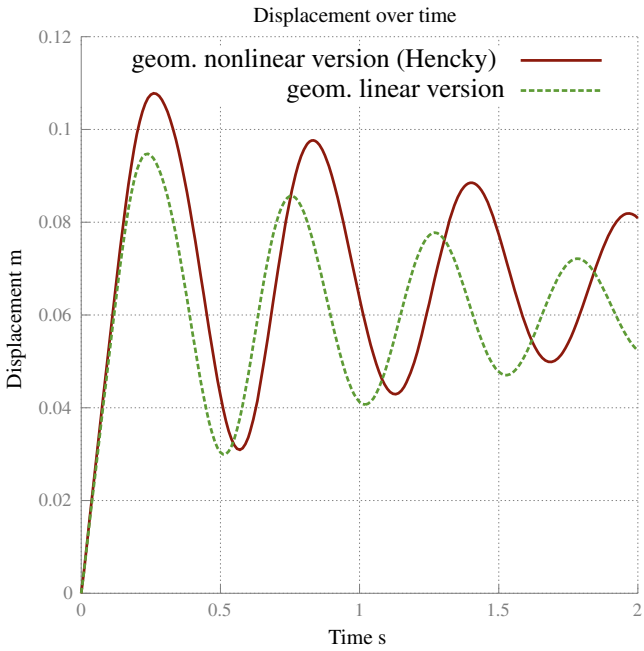
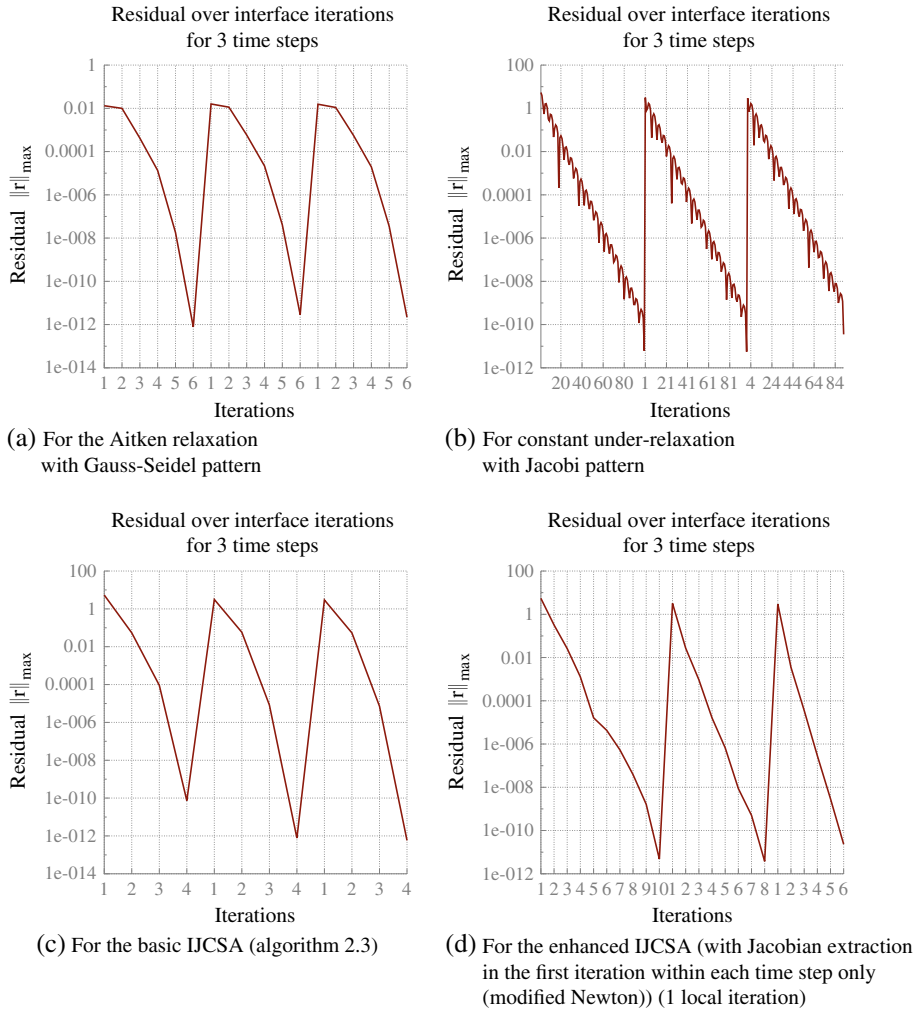


Figure 7. Numerical solution of the truss versus truss problem.

The numerical solution for the chosen parameter set is shown in Figure 7. The numerical dissipation of the BE time integrator is dominant. However, this is the exact monolithic solution of the problem setting. Additionally, Figure 7 shows the geometrical linear solution of the truss versus truss problem. By comparing that to the geometrical nonlinear (Hencky) solution of the truss versus truss problem, it is evident that the geometrical nonlinearity changes structural response distinctly.

DOI: 10.1002/nme



(a) For the Aitken relaxation with Gauss-Seidel pattern

(b) For constant under-relaxation with Jacobi pattern

(c) For the basic IJCSA (algorithm 2.3)

(d) For the enhanced IJCSA (with Jacobian extraction in the first iteration within each time step only (modified Newton)) (1 local iteration)

Figure 8. Convergence behavior of the truss versus truss problem.

The fastest IJCSA version is about 18 times faster than Aitken relaxation for the GS pattern (this takes also the extraction of the Jacobian into account). Moreover, GS-based patterns are no option for a general multi-code co-simulation scenario: they have the severe disadvantage of data flow dependency of all participating subsystems.

Figure 8 shows the evolution of the interface residual norm for the interface iterations needed to accomplish three time steps. The Aikten version shows fairly good reduction of the interface residual. It needs six iterations to render the residual below $1.0E-10$. The constant under-relaxation shows a linear convergence rate, which leads to a large number of iterations (approx. 100 per time step). The basic version of the IJCSA needs the least number of iterations. The modified Newton starts with 10 iterations and drops down to six iterations for the third time step. The modified Newton method is best when the problem is 'mildly' nonlinear; there, it is more efficient than a full Newton method.

4.2. A multi-code problem

In the following a co-simulation with five subsystems is carried out by using the IJCSA. The subsystems $\mathcal{S}_1$ and $\mathcal{S}_2$ are thereby modeled by an nonlinear operator described in Subsection 4.1.4. The spring-subsystem $\mathcal{S}_3$ is a simple linear spring model. The dashpot-subsystem $\mathcal{S}_4$ is integrated with the BE integrator; this subsystem is also linear. The last subsystem is a nonlinear ODE

DOI: 10.1002/nme

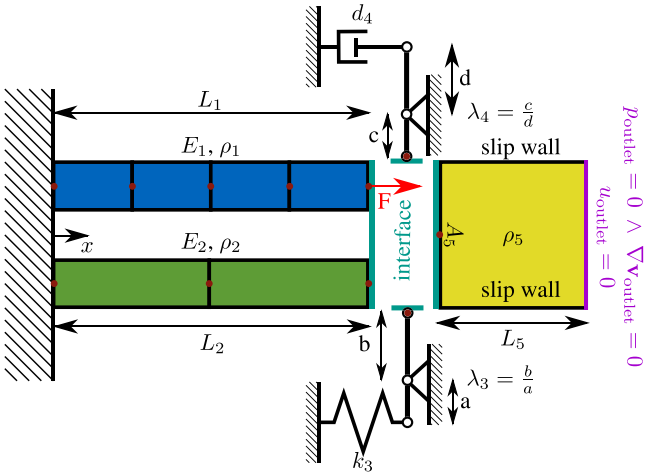


Figure 9. Setup of the multi-code problem.

Table IV. Input/output quantities for the multi-code problem.

Subsystem	Input U	Output Y
$\mathcal{S}_1$	f_1	u_1
$\mathcal{S}_2$	f_2	u_2
$\mathcal{S}_3$	f_3	u_3
$\mathcal{S}_4$	f_4	u_4
$\mathcal{S}_5$	u_5	f_5

stemming from the simplification of the Navier–Stokes equations (NSEs) on moving grids; it is integrated with the trapezoidal rule. The problem setup is illustrated in Figure 9.
The input/output quantities for the multi-code problem are presented in Table IV.

The following interface operators and interface residuals are defined for this problem:

$$\mathcal{R}_1 = \mathcal{I}_1(U_1, U_2, U_3, U_4, Y_5) = U_1 + U_2 + \frac{1}{\lambda_3}U_3 + \frac{1}{\lambda_4}U_4 - Y_5 = 0 \tag{110}$$

$$\mathcal{R}_2 = \mathcal{I}_2(\mathcal{S}_1(U_1), \mathcal{S}_2(U_2)) = Y_1 - Y_2 = 0 \tag{111}$$

$$\mathcal{R}_3 = \mathcal{I}_3(\mathcal{S}_1(U_1), \mathcal{S}_3(U_3)) = Y_1 + \lambda_3 Y_3 = 0 \tag{112}$$

$$\mathcal{R}_4 = \mathcal{I}_4(\mathcal{S}_1(U_1), \mathcal{S}_4(U_4)) = Y_1 + \lambda_4 Y_4 = 0 \tag{113}$$

$$\mathcal{R}_5 = \mathcal{I}_5(\mathcal{S}_1(U_1), \mathcal{S}_5(U_5)) = Y_1 - U_5 = 0 \tag{114}$$

Hence, the interface Jacobian matrix is given by

$$\begin{bmatrix} 1 & 1 & \frac{1}{\lambda_3} & \frac{1}{\lambda_4} & -\frac{\partial \mathcal{S}_5}{\partial U_5} \\ \frac{\partial \mathcal{S}_1}{\partial U_1} & -\frac{\partial \mathcal{S}_2}{\partial U_2} & 0 & 0 & 0 \\ \frac{\partial \mathcal{S}_1}{\partial U_1} & 0 & \lambda_3 \frac{\partial \mathcal{S}_3}{\partial U_3} & 0 & 0 \\ \frac{\partial \mathcal{S}_1}{\partial U_1} & 0 & 0 & \lambda_4 \frac{\partial \mathcal{S}_4}{\partial U_4} & 0 \\ \frac{\partial \mathcal{S}_1}{\partial U_1} & 0 & 0 & 0 & -1 \end{bmatrix} \tag{115}$$

DOI: 10.1002/nme

4.2.1. Subsystem 1 - truss 1. This subsystem is defined in Subsection 4.1.4.

4.2.2. Subsystem 2 - truss 2. This subsystem is defined in Subsection 4.1.4.

4.2.3. Subsystem 3 - spring. A linear spring model is used in subsystem 3, which is given by

$$u_3^{n+1} = \frac{1}{k_3} f_3^{n+1}. \tag{116}$$

Hence, the interface Jacobian for this subsystem is constant

$$\frac{\partial \mathcal{S}_3}{\partial U_3} = \frac{1}{k_3}. \tag{117}$$

4.2.4. Subsystem 4 - dashpot. In subsystem 4, a linear dashpot model is used

$$\dot{u}_4 d_4 = f_4, \tag{118}$$

which is integrated by using the BE integrator

$$u_4^{n+1} = u_4^n + \frac{h}{d_4} f_4^{n+1}. \tag{119}$$

The interface Jacobian for this subsystem is constant for a constant time step and is given by

$$\frac{\partial \mathcal{S}_4}{\partial U_4} = \frac{h}{d_4}. \tag{120}$$

4.2.5. Subsystem 5 - idealized piston. This section derives the governing equations for the idealized piston subsystem. As the fluid is modeled incompressibly, the starting point for the derivations are the NSEs for incompressible flow in combination with a linear material law (Stokes relations). These equations describe the flow on a fixed grid (Eulerian point of view). If the structure will be modeled in the Lagrangian framework, it would cause a problem at the interface of fluid and structure. In order to cope with this problem, the ALE method is used. As the ALE method is a hybrid reference frame (mixture of Eulerian and Lagrangian reference frame), the NSEs need to be changed [12] to

$$\frac{\partial v_i}{\partial x_i} = 0 \tag{121}$$

$$\frac{\partial v_i}{\partial t} + \left(v_j - v_j^{\text{grid}}\right) \frac{\partial v_i}{\partial x_j} = -\frac{1}{\rho_5} \frac{\partial p}{\partial x_i} + \nu \frac{\partial^2 v_i}{\partial x_j^2} \tag{122}$$

where ρ_5 is the fluid density and ν is the kinematic viscosity. p is the most important quantity for the idealized piston subsystem as it represents the pressure. The flow velocity components are denoted by v_i. Furthermore, v_j^{grid} are the so-called grid velocity components and $v_j - v_j^{\text{grid}}$ the convective velocity components (note that grid displacement components are denoted via $u_j^{\text{grid}} = u_j$). By choosing an appropriate set of boundary conditions (Figure 9), the fluid domain can be reduced to a one-dimensional setting; hence, the equation set (121) and (122) reduces to

$$\frac{\partial v_1}{\partial x_1} = 0 \tag{123}$$

$$\frac{\partial v_1}{\partial t} + \left(v_1 - v_1^{\text{grid}}\right) \frac{\partial v_1}{\partial x_1} = -\frac{1}{\rho_5} \frac{\partial p}{\partial x_1} + \nu \frac{\partial^2 v_1}{\partial x_1^2} \tag{124}$$

DOI: 10.1002/nme

These equations can be reformulated to one integrable ODE for the pressure [if equation (123) is inserted into equation (124)]:

$$\frac{\partial v_1}{\partial t} = -\frac{1}{\rho_5}\frac{\partial p}{\partial x_1} \rightarrow \frac{\partial p}{\partial x_1} = -a_1\rho_5 \tag{125}$$

with a, being the acceleration in x-direction. In the subsequent derivations, the index is dropped as only one dimension is present. Equation (125) may be solved for the pressure in the following manner:

$$\int \frac{\partial p}{\partial x}\mathrm{d}x = -\int a\rho_5 \mathrm{d}x \tag{126}$$

$$p(x) = -a\rho_5 x + C \tag{127}$$

The integration constant C can be determined from the outlet boundary condition for the pressure $p(x)|_{x=L_5} = 0$. Please note that the coordinate system in Figure 9 is fixed in space. As result, one obtains the following linear function for the pressure distribution

$$p(x) = a\rho_5(L_5 - x). \tag{128}$$

The ordinary differential nonlinear equation for subsystem 5 if given by

$$\ddot{u}_5(\rho_5 \cdot A_5 \cdot L_5) - \ddot{u}_5 u_5(\rho_5 \cdot A_5) = f_5$$

with u_5 being the interface displacement and f_5 the interface force.

This equation is integrated with the help of the trapezoidal rule:

$$\dot{u}_5^{n+1} = \frac{2}{h}\left(u_5^{n+1} - u_5^n\right) - \dot{u}_5^n. \tag{129}$$

Thus, the time discretized equation is given by

$$\begin{aligned}
&\begin{bmatrix} 0 & -1 & 0 \\ -1 & \left(L_5 - u_5^{n+1}\right)\frac{2\rho_5 A_5}{h} & 0 \\ 0 & \frac{2}{h} & -1 \end{bmatrix}\begin{bmatrix} f_5^{n+1} \\ v_5^{n+1} \\ a_5^{n+1} \end{bmatrix} \\
&= \begin{bmatrix} \frac{2}{h} & 1 & 0 \\ 0 & \left(L_5 - u_5^{n+1}\right)\frac{2\rho_5 A_5}{h} & \left(L_5 - u_5^{n+1}\right)\rho_5 A_5 \\ 0 & \frac{2}{h} & 1 \end{bmatrix}\begin{bmatrix} u_5^n \\ v_5^n \\ a_5^n \end{bmatrix} - \begin{bmatrix} \frac{2}{h}u_5^{n+1} \\ 0 \\ 0 \end{bmatrix}.
\end{aligned} \tag{130}$$

With that the interface Jacobian of subsystem 5 is

$$\frac{\partial \mathcal{S}_5}{\partial U_5} = \frac{2\rho_5 A_5}{h}\left(\frac{2}{h}u_5^n - \frac{4}{h}u_5^{n+1} + 2v_5^n + \frac{h}{2}a_5^n + \frac{2L_5}{h}\right). \tag{131}$$

4.2.6. Results. The spacial discretization is the same for subsystems 1 and 2 as in the previous example. The simulation time is 1s, this results in 1000 time steps. Two different settings are simulated (Table V):

As in setting 1, the fluid density ρ_5 is almost zero, the solution is verified with the monolithic solution of all structural subsystems (one to four) performed in Abaqus. For setting 2, the fluid density is increased, and the added mass effect becomes dominant. For this highly nonlinear example, the IJCSA needs in average 3.7 interface iterations. Please note that the interface residual tolerance of the previous example is used (Equation (108)). If the modified Newton version of the IJCSA is used, the interface iteration count rises to five in average per time step. A comparison to classical techniques is not possible as all of them failed to deliver a converged solution for this problem. The results are demonstrated in Figure 10.

DOI: 10.1002/nme

Table V. Parameter sets for the multi-code problem.

Parameter	Values for setting 1	Values for setting 2
h	0.001 s	0.001 s
F	12.25 N	12.25 N
E_1	20 N/m^2	20 N/m^2
ρ_1	0.55 kg/m^3	0.55 kg/m^3
L_1	1 m	1 m
E_2	50 N/m^2	50 N/m^2
ρ_2	0.5 kg/m^3	0.5 kg/m^3
L_2	1 m	1 m
k_3	500 N/m	500 N/m
λ_3	2−	2−
d_4	10 Ns/m	10 Ns/m
λ_4	2−	2−
A_5	0.1 m^2	0.1 m^2
L_5	1m	1m
ρ_5	1.0 $E-10$ kg/m^3	2.75 kg/m^3

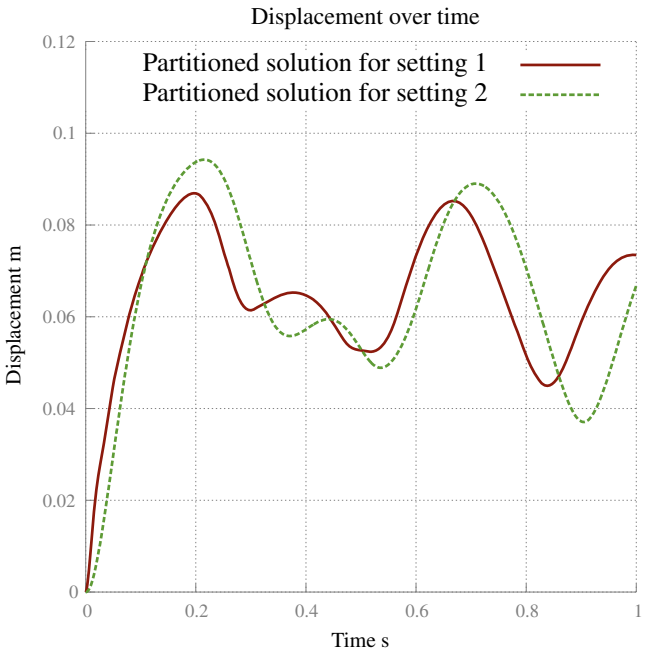


Figure 10. Numerical solution of the multi-code problem.

This example demonstrates that the IJCSA can handle complex co-simulation scenarios in an efficient and accurate manner. It can handle different numerical time integrators at the different subsystems.

Note that Figure 11 shows the absolute residual. In a general implementation, it is better to use a relative residual. For the discussion of the multi-code problem, there is no difference of using the absolute or relative residual as the applied force and the maximal interface displacement are close enough to one. Hence, by using them in a relative definition will not change the outcome of the discussion.

Figure 11 shows that for the multi-code problem, the different residuals are not equally important. Here, the residual associated with the interface forces ($\mathcal{R}_1$) plays the dominant role. In a general co-simulation scenario, this is always the case. The advantage of using a parallel (JC) pattern is that

DOI: 10.1002/nme

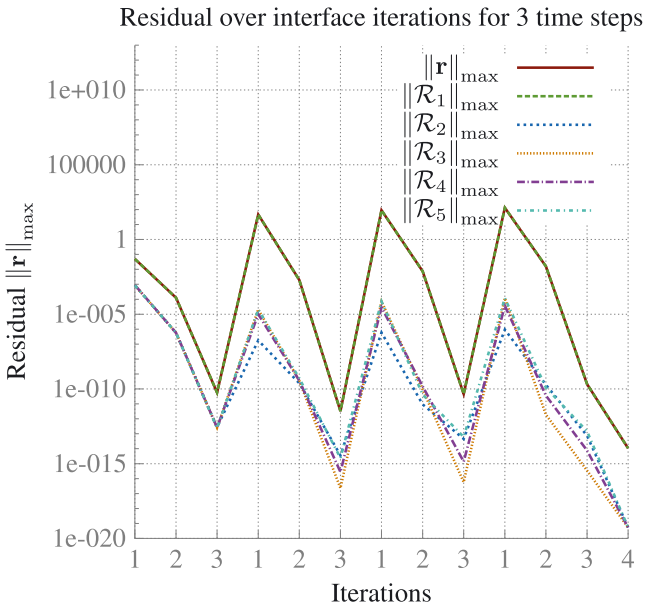


Figure 11. Residual evolution for the different components of the interface residual vector for setting 2 of the multi-code problem.

this exposes all interface quantities in the interface residual vector. This is not the case if a GS data flow is used; for example, it can be seen in the stability section that GS removes the force from the interface residual. Hence, the advantages of the IJCSA is that it controls all residuals that is will lead to more accurate results.

5. CONCLUSION AND SUMMARY

In this work, we present a co-simulation algorithm that is based on the idea of an interface Jacobian. Therefore, the algorithm is called the Interface Jacobian-based Co-Simulation Algorithm (IJCSA). With the help of the stability analysis, it is shown that the algorithm outperforms classical fixed-point techniques based on the Jacobi (JC) and Gauss-Seidel (GS) approaches. In the Example Section, numerical investigations are performed where the IJCSA is compared to classical co-simulation algorithms, which include Aitken relaxation for the GS pattern. The IJCSA is for all examples by far more stable and efficient than the classical techniques. Its design allows for parallel execution of all the involved subsystem. Furthermore, performance enhancements are discussed and investigated with different nonlinear examples. The IJCSA can cope with different time integrators in the subsystems. Algebraic loops [13] can be handled because the co-simulation is formulated in residual form. The IJCSA presents an efficient, accurate, and robust treatment for solving co-simulation scenarios.

APPENDIX A

A.1. System vectors and matrices for the truss versus truss example

A.1.1. Element internal force vector.

$$\mathbf{p}_e(\mathbf{u}) = EL_e \begin{bmatrix} \frac{\ln\left(\frac{L_e+u_2-u_1}{L_e}\right)}{L_e+u_2-u_1} \\ -\frac{\ln\left(\frac{L_e+u_2-u_1}{L_e}\right)}{L_e+u_2-u_1} \end{bmatrix} \tag{A.1}$$

Where u_1 is the displacement at element node 1 and u_2 the displacement at element node 2. The element length is denoted by L_e.

DOI: 10.1002/nme

A.1.2. Element tangent stiffness matrix.

$$\mathbf{K}_{\mathrm{e}}(\mathbf{u}) = EL_{\mathrm{e}} \begin{bmatrix} -\frac{\ln\left(\frac{L_{\mathrm{e}}+u_2-u_1}{L_{\mathrm{e}}}\right)-1}{(L_{\mathrm{e}}+u_2-u_1)^2} & \frac{\ln\left(\frac{L_{\mathrm{e}}+u_2-u_1}{L_{\mathrm{e}}}\right)-1}{(L_{\mathrm{e}}+u_2-u_1)^2} \\ \frac{\ln\left(\frac{L_{\mathrm{e}}+u_2-u_1}{L_{\mathrm{e}}}\right)-1}{(L_{\mathrm{e}}+u_2-u_1)^2} & -\frac{\ln\left(\frac{L_{\mathrm{e}}+u_2-u_1}{L_{\mathrm{e}}}\right)-1}{(L_{\mathrm{e}}+u_2-u_1)^2} \end{bmatrix} \tag{A.2}$$

A.1.3. Element mass matrix.

$$\mathbf{M}_{\mathrm{e}} = \rho L_{\mathrm{e}} \begin{bmatrix} \frac{1}{2} & 0 \\ 0 & \frac{1}{2} \end{bmatrix} \tag{A.3}$$

Note that this is the lumped form of the mass matrix.

ACKNOWLEDGEMENT

The first author would like to gratefully thank the Dassault Systemes Simulia Corp. HQ for the financial support.

REFERENCES

1. Farhat C, Lesoinne M. Two efficient staggered algorithms for the serial and parallel solution of three-dimensional nonlinear transient aeroelastic problems. *Computer Methods in Applied Mechanics and Engineering* 2000; **182**: 499–515.
2. Felippa CA, Park KC. Staggered transient analysis procedures for coupled mechanical systems: formulation. *Computer Methods in Applied Mechanics and Engineering* 1980; **24**:61–111.
3. Küttler U, Wall WA. Fixed-point fluid-structure interaction solvers with dynamic relaxation. *Computational Mechanics* 2006; **43**:61–72.
4. Schulte S. *Modulare und Hierarchische Simulation Gekoppelter Probleme*. VDI-Verlag, 1998.
5. Farhat C, van der Zee KG, Geuzaine P. Provably second-order time-accurate loosely-coupled solution algorithms for transient nonlinear computational aeroelasticity. *Computer Methods in Applied Mechanics and Engineering* 2006; **17**:1973–2001.
6. Hanke-Bourgeois M. *Grundlagen der Numerischen Mathematik und des Wissenschaftlichen Rechnens*. Teuber, 2001.
7. Joosten MM, Dettmer WG, Perić D. Analysis of the block Gauss–Seidel solution procedure for a strongly coupled model problem with reference to fluid-structure interaction. *International Journal for Numerical Methods in Engineering* 2009; **78**:757–778.
8. Deuflhard P. *Newton Methods for Nonlinear Problems: Affine Invariance and Adaptive Algorithms*. Springer, 2011.
9. Causin P, Gerbeau JF, Nobile F. Added-mass effect in the design of partitioned algorithms for fluid-structure problems. *Computer Methods in Applied Mechanics and Engineering* 2005; **42**:4506–4527.
10. Møller H. *Analysis and Optimization for Fluid-Structure Interaction Problems*. Aalborg University Denmark, 2002.
11. Irons BM, Tuck RC. A version of the Aitken accelerator for computer iteration. *International Journal for Numerical Methods in Engineering* 1969; **1**:275–277.
12. Donea J, Huerta A, Ponthot JP, Rodríguez-Ferran A. *Arbitrary Lagrangian–Eulerian Methods*, Encyclopedia of computational mechanics. Wiley Online Library, 2004.
13. Bastian J, Clauß C, Wolf S, Schneider P. Master for co-simulation using FMI. *8th International Modelica Conference*, Dresden, 2011.
14. Sobieszczanski-Sobieski J. On the sensitivity of complex, internally coupled systems, *NASA Technical Memorandum*, 1988.

DOI: 10.1002/nme